Chapter 3: Enhancing the User Interface

In this chapter, you will learn how to enhance basic user interface (UI) elements, such as menus and toolbars, and create new UI elements, such as modeless dialog boxes and dialog bars.

This chapter begins with techniques for enhancing and extending the UI capabilities of an MFC-based application. You will learn how to create dynamic menus, add cascading menus, and include graphics in menus and status bars.

You will also learn to extend the functionality of dialog boxes by managing Dialog Data Exchange (DDX) and Dialog Data Validation (DDV), extending DDV, and coordinating data between controls.

In the later sections of this chapter, you will learn how to create and manage modeless dialog boxes, dialog bars, and rebars.

Objectives

After completing this chapter, you will be able to:

- Create dynamic menus, cascading menus, and ownerdraw menus.
- Place dockable toolbars in an application.
- Include graphics and additional panes in status bars.
- Use and customize common dialog boxes.
- Extend DDV.
- Create tabbed dialog boxes and property sheets.
- Invoke and display modeless dialog boxes.
- Create dialog bars and rebars.

45 Enhancing Menus

Using the resource editor is the easiest way to create menus. However, you cannot use the resource editor to create menus at run time. Instead, you need to add the required code to your application.

The first topic in this section shows how to implement dynamic menus. Dynamic menus provide the user with an appropriate set of choices, depending on the context. They also offer you the choice of presenting the user with different options at run time.

The next topic covers how to implement cascading menus. Cascading menus enable you to provide the user with a level of depth that is easy to follow and understand.

The final topic discusses ownerdraw menus. Ownerdraw menus enable you to put bitmaps and graphical objects in a menu instead of just text. For example, you can create an ownerdraw menu to display color palettes and drawing objects in menus.

This section includes the following topics:

- Implementing Dynamic Menus
- Implementing Cascading Menus
- Implementing Ownerdraw Menus

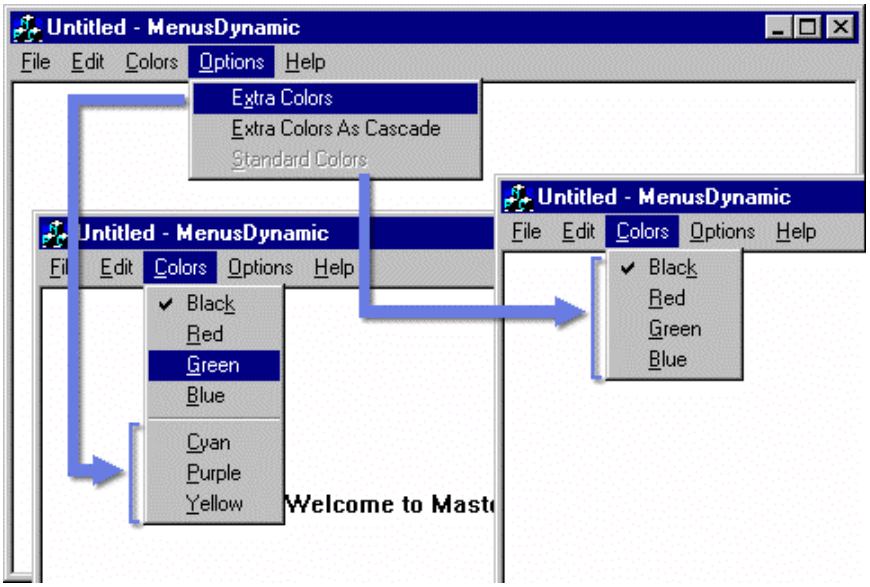
Notes

46 Implementing Dynamic Menus

Static menus are defined at design time using the resource editor. However, in some cases, you may want to provide context-sensitive menu options instead of making all menu items available to the user at all times. To provide menu options based on the context, you need to add and delete menus at run time.

Menus added at run time are named dynamic menus and are based on system, user, and data settings. To see an illustration of dynamic menus, click this icon.

 (CD-ROM displays the following illustration.)



To see a demonstration of how to create a dynamic menu, click this icon.

 (CD-ROM plays the demonstration, "Creating Dynamic Menus.")

Consolidating Command Handlers into a Command Range

Before creating a dynamic menu, you need to know how to consolidate a number of command handlers into a command range. This makes creating dynamic menus easier.

Because there are many menu items, there are also many command handlers. Often, command handlers are identical or nearly identical in their functionality. When this is the case, map all the messages with similar functionality to a single handler function. Message map ranges enable you to do this for a contiguous range of messages.

You can map ranges of command IDs to:

- A command handler function.
- A command update handler function.

You can map control notification messages for a range of control IDs to a message handler function.

Notes

▶ **To consolidate command handlers**

1. Determine the maximum number of new message entries, and add symbolic constants in the resource table for each message entry.
   a. On the **View** menu, click **Resource Symbols**.
   b. In the **Resource Symbols** dialog box, click **New**.
   c. In the **New Symbol** dialog box, specify a name and a value.
2. Repeat Step 1 to add resource symbols as needed. Choose consecutive numbers for resource symbol values, and set the initial number greater than 33000 to avoid conflicts with any predefined or existing messages. At a minimum, define a beginning and ending number.
3. Add an ON_COMMAND_RANGE entry to the message map of the class that will receive the commands from the dynamic menu. Specify the beginning and ending numbers of the range that you specified in the resource symbols table as the first two arguments of the macro. For example:

```
ON_COMMAND_RANGE(ID_COLOR_CYAN, ID_COLOR_YELLOW, OnColors)
```

Note ClassWizard does not support ON_COMMAND_RANGE entries. You need to add them manually.

4. Add the entry to the class's header file in the DECLARE_MESSAGE_MAP section as follows:

```
afx_msg void OnColors( UINT nID );
```

Adding a Dynamic Menu

To add a dynamic menu, you must first decide which top-level menu will display the dynamic menu. Menus are counted from left to right, starting at zero. Next, get a pointer to this top-level menu. You need to write a function to accept the commands that the dynamic menu generates. You will also need to write an update handler and an update UI function for the menu.

Notes

▶ **To add a dynamic menu**

1. Add entries to the string table to store the caption of each item on the dynamic menu.
2. Write the code that will add the menu items.

   To see sample code that shows how to add menu items, click this icon.
    (CD-ROM displays the following sample code.)

```
// Creating a menu at runtime
// Samples\CH03\Dynamenu
void CMenusDynamicView::OnOptionsExtracolors()
{
    CMenu *pAddinMenu, *pTopMenu;

    // To append to the Colors menu, we'll need a pointer
    // to it. We begin by obtaining a pointer to the top-
    // level menu.
    pTopMenu = AfxGetMainWnd()->GetMenu();

    // Colors is the 3rd menu, but that's #2 in a 0-based
    // system.
    pAddinMenu = pTopMenu->GetSubMenu(2);
    ASSERT(pAddinMenu != NULL);

    // First, add a separator to separate the default
    // menus from the dynamic menus.
    pAddinMenu->AppendMenu(MF_SEPARATOR);

    // Append the 3 menu items. The captions for the menu commands are
    // stored in the string table. .
    CString prompt;
    for (int i = 0; i < 3; i++)
    {
        prompt.LoadString(ID_COLORS_CYAN + i);
        pAddinMenu->AppendMenu(MF_STRING,
                        ID_COLORS_CYAN + i, prompt);
    }
}
```

Notes

3. Write the function that will accept the commands that are generated by the additional menus.

   To see sample code that shows how to write the command handler for a dynamic menu, click this icon.  (CD-ROM displays the following sample code.)

```
//Generic command handler for dynamic menu items
//Samples\CH03\Dynamenu
void CMenusDynamicView::OnColors(UINT nID)
{
    CMenusDynamicDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    pDoc->SetColor(IDtoColorRef(nID));
    Invalidate();
}
// This function converts one of the 7 Command IDs to a
// COLORREF value.
COLORREF CMenusDynamicView::IDtoColorRef(int nID)
{
    switch (nID)
    {
        case ID_COLORS_RED:
            return RED;
        case ID_COLORS_GREEN:
            return GREEN;
        case ID_COLORS_BLUE:
            return BLUE;
        case ID_COLORS_CYAN:
            return CYAN;
        case ID_COLORS_PURPLE:
            return PURPLE;
        case ID_COLORS_YELLOW:
            return YELLOW;
        default:
            return BLACK;
    }
}
```

4. If you want an update handler for the menus you have just added, add another entry to the class's message map. For example:

```
ON_UPDATE_COMMAND_UI_RANGE(ID_COLORS_CYAN, ID_COLORS_YELLOW, OnUpdateColors)
```

5. Add an entry to the header file for the class, as shown in this example:

```
afx_msg void OnUpdateColors(CCmdUI* pCmdUI);
```

Notes

6. Finally, write the update function, as shown in this example:

```
void CMenusDynamicView::OnUpdateColors(CCmdUI* pCmdUI)
{
    CMenusDynamicDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    pCmdUI->SetCheck(pDoc->GetColor() == IDtoColorRef(pCmdUI->m_nID));
}
```

The code samples discussed in this topic appear in the sample application Dynamenu, which is located in the folder \Samples\Ch03.

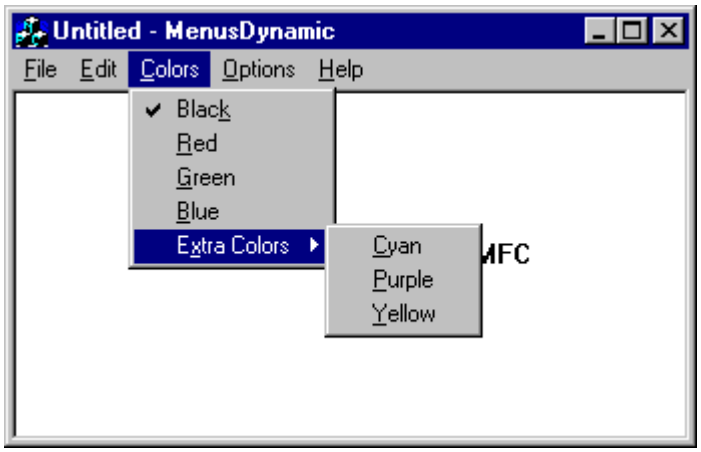
Notes

47 Implementing Cascading Menus

A cascading menu is a pop-up menu that appears when the user points to an item on a menu that has already been invoked. Cascading menus are useful because they display menu items hierarchically.

There is no actual limit to the number of menus that you can nest by cascading, but user interface guidelines suggest that you use no more than four levels of nesting.

The following illustration shows a cascading menu.



You create cascading menus at design time by using the resource editor. To implement cascading menus at run time, you need to add code to your application and use the **AppendMenu** function.

Notes

To see sample code that shows how to add a cascading menu at run time, click this icon.
 (CD-ROM displays the following sample code.)

```
// Creating a cascading menu and adding menu items to it
// Samples\CH03\Dynamenu
void CMenusDynamicView::OnOptionsExtracolorsCascade()
{
    // A cascading menu is a new menu object that must
    // persist after this function terminates. Thus the
    // use of the pointer m_pExtraColors, which is a member
    // of the view class. It's needed elsewhere in this
    // class to properly delete the object allocated in
    // this function.
    m_pExtraColors = new CMenu;
    m_pExtraColors->CreatePopupMenu();

    // Next, add existing menu items to this newly
    // created menu object.
    CString prompt;
    for (int i = 0; i < 3; i++)
    {
        prompt.LoadString(ID_COLORS_CYAN + i);
        m_pExtraColors->AppendMenu(MF_STRING,
                ID_COLORS_CYAN + i, prompt);
    }

    CMenu *pTopMenu, *pSubMenu;

    pTopMenu = AfxGetMainWnd()->GetMenu();
    pSubMenu = pTopMenu->GetSubMenu(2);

    // Now that we have a pointer to the appropriate sub
    // menu, the new menu object can be appended. The
    // flag MF_POPUP tells AppendMenu how to interpret
    // the 3rd argument.
    prompt.LoadString(ID_EXTRA_COLORS);
    pSubMenu->AppendMenu(MF_STRING | MF_POPUP,
                (UINT)m_pExtraColors->m_hMenu, prompt);

    // Be sure to examine the code in
    // OnOptionsStandardcolors() to see how this menu
    // object is deleted, NOT removed!Also, examine
    // OnDestroy.
}
```

Your application must include code to delete cascading menus created at run time. Call the **DeleteMenu** function to delete menus at run time.

Notes

To see sample code that shows how to delete a cascading menu, click this icon.
 (CD-ROM displays the following sample code.)

```
// Destroying a cascading menu and updating the view
// Samples\CH03\Dynamenu
void CMenusDynamicView::OnOptionsStandardcolors()
{
    CMenu *pAddinMenu, *pTopMenu;
    // To remove items from the Colors menu, we'll need a
    // pointer to it. We begin by obtaining a pointer to
    // the top-level menu.
    pTopMenu = AfxGetMainWnd()->GetMenu();

    // Colors is the 3rd menu, but that's #2 in a 0-based
    // system.
    pAddinMenu = pTopMenu->GetSubMenu(2);

    int i = pAddinMenu->GetMenuItemCount();

    // The cascading menu must be properly destroyed,
    // since it was allocated using CreatePopupMenu in
    // another function.

    // First delete the cascading menu. It's also
    // necessary to free the object that was allocated in
    // OnOptionsExtracolorsCascade.

    pAddinMenu->DeleteMenu(4, MF_BYPOSITION);
    delete m_pExtraColors;
    m_pExtraColors = 0;

    // Finally, set the color to black and force a
    // redraw;
    CMenusDynamicDoc* pDoc = GetDocument();
    pDoc->SetColor(BLACK);
    Invalidate();
}
```

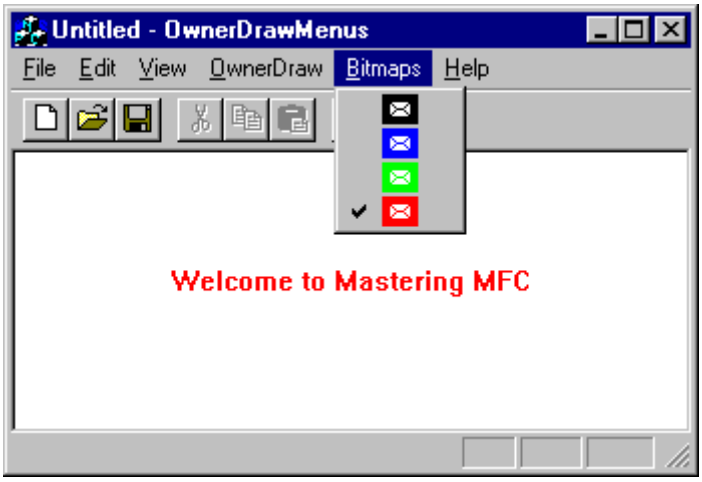
The code samples discussed in this topic appear in the sample application Dynamenu, which is located in the folder \Samples\Ch03.

Notes

48 Implementing Ownerdraw Menus

You can enhance a menu by displaying a bitmap or some other graphical object in place of text. For example, instead of displaying the word "Red" in a menu, you can display a rectangle that is the color red. To do this, you need to use an ownerdraw menu.

The following illustration shows an ownerdraw menu.



▶ **To create an ownerdraw menu**

1. Add a new class derived from **CMenu**. For example:

   ```
   class CODmenus : public CMenu
   ```

2. Add a **CODmenus** object to the **CMainFrm** class in an SDI application (or to the **CChildFrame** class in an MDI application). For example:

   ```
   CODmenus m_ownerDrawMenu;
   ```

3. Define a function that will create and attach the ownerdraw menu to the menu bar. Call this function from the **OnCreate** function of the frame class.

Notes

To see sample code that shows how to create and attach an ownerdraw menu, click this icon.
 (CD-ROM displays the following sample code.)

```
// Function to create and add an ownerdraw menu
//Samples\CH03\Owndraw
void CMainFrame::AttachOwnerDrawMenu()
{
    // Append 4 menus to the empty menu object m_ownerDrawMenu
    // (which is an object of a custom class and is a member
    // of the frame class).  Each appended menu is owner drawn.
    for (int iColor = 0; iColor <= (IDR_COLOR_LAST-IDR_COLOR_FIRST);
iColor++)
    {
        // Skip Cyan, which is the 3rd color.
        if (3 == iColor) continue;

        // 3 bit encoded RGB values
        BYTE red = (BYTE)(((iColor & 4) != 0) * 255);
        BYTE green = (BYTE)(((iColor & 2) != 0) * 255);
        BYTE blue = (BYTE)((iColor & 1) * 255);

        // Each appended menu has an RGB value associated with it
        // that will be passed to DrawItem in the itemData variable.
        // See DrawItem.
        m_ownerDrawMenu.AppendMenu(MF_ENABLED | MF_OWNERDRAW,
                IDR_COLOR_FIRST + iColor, (LPCTSTR)RGB(red, green, blue));
    }

    // Replace the specified menu item with a color popup
    //  (note: will only work once)
    CMenu* pMenuBar = GetMenu();
    ASSERT(pMenuBar != NULL);

    TCHAR szString[256];
    // Colors is the 4th (3rd from 0) menu. We want its string.
    pMenuBar->GetMenuString(3, szString, sizeof(szString),
                            MF_BYPOSITION);

    // Make the newly created menu become the 4th menu's popup.
    pMenuBar = GetMenu();
    ASSERT(pMenuBar != NULL);
    pMenuBar->ModifyMenu(3, MF_BYPOSITION | MF_POPUP,
                        (UINT)m_ownerDrawMenu.m_hMenu, szString);
}
```

Notes

4. Write a command handler to implement the ownerdraw menu items.
5. Write update UI handlers for the menu items.

The sample application Owndraw shows how to implement the functionality described above, and also shows how to place a bitmap in a menu. In the sample application, the menu items are sized and drawn. The application is located in the folder \Samples\ChO3.

Notes

49 Enhancing Toolbars

When you use MFC AppWizard to create an application, you specify whether you want the default toolbar (IDR_MAINFRAME) added to your MFC-based application. If you choose the default toolbar, the MFC framework will add the toolbar and the default functionality.

This section describes how to enable docking for the default toolbar and create floating toolbars.

This section includes the following topics:

- Enabling Docking in a Frame Window
- Creating a Floating Toolbar

Notes

50 Enabling Docking in a Frame Window

The Microsoft Foundation Class (MFC) Library supports dockable toolbars. A dockable toolbar can be attached, or "docked," to any side of its parent window, or it can float in its own miniframe window (by using **CMiniFrameWnd**). The following illustration shows a floating toolbar.



If you use MFC AppWizard to generate the skeleton of your application, you can choose to use dockable toolbars. By default, MFC AppWizard generates the code to perform the following actions necessary to place a dockable toolbar in your application:

- Enable docking for the frame window.
- Enable docking for the toolbar.
- Dock the toolbar to the frame window.

If the code for any of these actions is missing, your application displays a standard toolbar. The last two actions must be performed for each dockable toolbar in your application.

To enable docking, use the **CFrameWnd::EnableDocking** function. This function takes a DWORD parameter, which is a set of style bits indicating which side of the frame window accepts docking. If a toolbar is about to be docked and there are multiple sides that can accept it, the sides are indicated in the following order: top, bottom, left, right. If you want to be able to dock control bars anywhere in the frame window, pass the CBRS_ALIGN_ANY parameter to the **EnableDocking** function.

Notes

51 Creating a Floating Toolbar

Floating toolbars are displayed in their own window, and they float on the parent window.

To see a demonstration of how to create a floating toolbar, click this icon.
 (CD-ROM plays the demonstration, "Creating a Floating Toolbar.")

▶ **To add a floating toolbar to your application**

1. Make certain that there is a command ID and a handler for every button that you place on your toolbar. The easiest way to do this is to create a menu selection for each toolbar button.
2. Use the resource editor to create a new toolbar resource. Associate each toolbar button with the ID of the command that it generates.
3. Add a menu item that will contain the code that makes the toolbar visible or invisible. Usually, you place the code in the **View** menu provided by MFC.
4. In your application's **CMainFrame** class header file, add a protected variable of type **CToolBar** as shown in the following example. This variable may be an object or a pointer. The type of variable you choose will determine the code you need to write in the frame's implementation file.

   ```
   protected:
       CToolBar * m_pColorToolbar;
   ```

5. If you made the variable in the previous step a pointer, set it to zero in the constructor for **CMainFrame**. For example:

   ```
   CMainFrame::CMainFrame()
       :m_pColorToolbar(0)
   {
   }
   ```

6. Use ClassWizard to add a command handler for the menu item that you created in Step 3. Place this handler in the **CMainFrame** class.
7. Write the code for the menu handler. The same command handler can be used to hide the control bar, which will make it function exactly like the default **CMainFrame** toolbar. You need to allow for three possible situations:
   - The code is called before the toolbar is created.
   - The code is called to hide the toolbar.
   - The code is called to restore the toolbar.

   The string table stores the error message strings.

Notes

To see sample code that shows how to handle these situations, click this icon.
 (CD-ROM displays the following sample code.)

```
// A generic command handler to create, show, and hide a floating
// toolbar
// Samples\CH03\Toolbar
void CMainFrame::OnViewColortoolbar()
{
    // The very first time, the toolbar needs to be
    // created. m_pColorToolbar is a pointer that is a
    // member of CMainFrame. See OnDestroy for the code
    // that deletes the object allocated here.
    if (0 == m_pColorToolbar)
    {
        m_pColorToolbar = new CToolBar;
        CString ErrMsg;
        if (0 == m_pColorToolbar->Create(this))
        {
            ErrMsg.LoadString(IDS_COLORTB_CREATE);
            ::AfxMessageBox(ErrMsg);
            return;
        }
        if (0 ==
        m_pColorToolbar->LoadToolBar(IDR_COLOR_TOOLBAR))
        {
            ErrMsg.LoadString(IDS_COLORTB_LOAD);
            ::AfxMessageBox(ErrMsg);
            return;
        }
        m_pColorToolbar->EnableDocking(CBRS_ALIGN_ANY);

        DockControlBar(m_pColorToolbar);
    }
    else
        // If the window is visible, hide it.
        if(m_pColorToolbar->IsWindowVisible() == TRUE)
            ShowControlBar(m_pColorToolbar, FALSE, FALSE);
        else
            // Otherwise, show it.
            ShowControlBar(m_pColorToolbar, TRUE, FALSE);
}
```

Notes

8. Add a UI handler to **CMainFrame** for the menu.

   To see sample code that shows how to write this UI handler, click this icon.
   
   (CD-ROM displays the following sample code.)

```
// Samples\CH03\Toolbar
void CMainFrame::OnUpdateViewColortoolbar(CCmdUI* pCmdUI)
{
    // Initially, the color toolbar pointer is 0, so the
    // UI handler must not call IsWindowVisible unless
    // the toolbar has been created.
    if (0 == m_pColorToolbar)
        pCmdUI->SetCheck(FALSE);
    else
        if(m_pColorToolbar->IsWindowVisible() == TRUE)
            pCmdUI->SetCheck(TRUE);
        else
            pCmdUI->SetCheck(FALSE);
}
```

9. If you created a pointer for the toolbar as shown in the sample code, use ClassWizard to add a handler for the WM_DESTROY message, and then delete the object that you created earlier.

   To see sample code that shows how to delete the object, click this icon.
   
   (CD-ROM displays the following sample code.)

```
//Samples\CH03\Toolbar
void CMainFrame::OnDestroy()
    CFrameWnd::OnDestroy();

    // Since the color toolbar was dynamically created,
    // it must be destroyed when the frame is destroyed.
    if (0 != m_pColorToolbar)
    {
        delete m_pColorToolbar;
        m_pColorToolbar = 0;

    }
}
```

10. Repeat the above process for additional toolbars.

The code samples discussed in this topic appear in the sample application Toolbar, which is located in the folder \Samples\Ch03.

Notes

52 Enhancing Status Bars

Although you typically display text in status bar panes, you can also display graphics in the panes.

▶ **To enable graphics in a status bar pane**

1. Derive a new class based on **CStatusBarCtrl** as follows:

```
class CCustomStatusBar : public CStatusBarCtrl
```

2. Add a public member variable in MainFrm.h as follows:

```
CCustomStatusBar    m_wndStatusBar;
```

3. From **CMainFrame::OnCreate**, invoke **CStatusBarCtrl::Create** to create the status bar and establish its properties. In the call to **CStatusBarCtrl::Create**, pass AFX_IDW_STATUS_BAR as the fourth argument. This will make your status bar's pane 0 function exactly like the default status bar's pane 0. In other words, it will show menu prompts:

```
if (!m_wndStatusBar.Create(
    WS_VISIBLE | WS_CHILD | CCS_BOTTOM | SBARS_SIZEGRIP,
    rcSB, this, AFX_IDW_STATUS_BAR))
{
    TRACE0("Failed to create status bar\n");
    return FALSE;
}
```

4. Override **CStatusBarCtrl::DrawItem**. This function renders the panes that have the **Ownerdraw** property.

Notes

To see sample code that shows how to override **CStatusBarCtrl::DrawItem**, click this icon. (CD-ROM displays the following sample code.)

```
// Called when the status bar needs repainting
// Samples\CH03\CustSBar
void CCustomStatusBar::DrawItem(LPDRAWITEMSTRUCT lpdis)
{
    // Handle drawing of the ownerdraw pane(s) of the
    // CGraphicStatusBar object.

    CDC dc;
    dc.m_hDC = lpdis->hDC;

    // The parameter gives us a rectangle that needs to be drawn
    CRect rect(lpdis->rcItem);

    // itemData gives us a pointer to a structure that contains
    // the ID of the bitmap that is to be drawn in the current.
    // This structure was originally filled by CMainFrame, and
    // then changed by the View's event handler that modifies
    // the displayed color.
    CBitmap Bitmap;
    PaneTool *pt = (PaneTool *)lpdis->itemData;
    Bitmap.LoadBitmap(pt->bitmapID);

    CDC srcDC;
    srcDC.CreateCompatibleDC(NULL);
    srcDC.SelectObject(&Bitmap);
    dc.BitBlt(rect.left, rect.top, rect.Width(), rect.Height(),
                &srcDC, 0, 0, SRCCOPY);
}
```

5. Override **CStatusBarCtrl::OnSize**. This function establishes the number, type, and sizes of panes in the status bar. Use **SetText** and the **SBT_OWNERDRAW** property for any panes for which you want to assign graphics.

Notes

To see sample code that shows how to override **CStatusBarCtrl::OnSize**, click this icon. (CD-ROM displays the following sample code.)

```
//Creating the panes of the statusbar
// Samples\CH03\CustSBar
void CCustomStatusBar::OnSize(UINT nType, int cx, int cy)
{
    CStatusBarCtrl::OnSize(nType, cx, cy);

    int aWidths[5] = {cx / 2, cx / 2 + 30, cx / 2 + 100,
                      cx / 2 + 130, -1};
    SetParts(5, aWidths);

    // Pane 0's text is written by the framework for menu and
    // toolbar status, so nothing needs to be done. The framework
    // will only do this if the custom status bar is given the ID
    // AFX_IDW_STATUS_BAR in the call to Create.

    // Pane 1 shows the status of the Caps Lock key.
    // Also See CMainFrame::OnCmdMsg
    DetermineKeyboardState();

    // Pane 2 has static text.
    DetermineTextColor();

    // Pane 3 has a bitmap placed in it. Thus it has to have
    // owner draw. We pass it the address of a structure that
    // contains the bitmap's ID.
    CMainFrame * cmf = (CMainFrame *)GetParent();
    SetText((LPCTSTR) &cmf->pt, 3, SBT_OWNERDRAW);

    // Pane 4 will, width -1, takes over the rest of the
    // status bar and contains the sizing grip.
}

void CCustomStatusBar::DetermineTextColor()
{
    CMainFrame * cmf = (CMainFrame *)GetParent();
    CCustomStatusBarDoc * ccsbd =
        (CCustomStatusBarDoc *)cmf->GetActiveDocument();

    CString color = "Black";
```

(Code continues on following page.)

Notes

```
    // If the document doesn't exist yet, just set the color
    // to black.
    if (0 != ccsbd)
        // Convert an RGB value to a CString.
        switch (ccsbd->GetColor())
        {
            case RED:
                color = "Red";
                break;
            case GREEN:
                color = "Green";
                break;
            case BLUE:
                color = "Blue";
                break;
        }

    SetText(color, 2, 0);
}

void CCustomStatusBar::DetermineKeyboardState()
{
    // We completely take over writing the word CAP.
    BYTE ks[256];

    GetKeyboardState(ks);

    CRect rect;
    GetRect(1, &rect);
    rect.OffsetRect(3, 2);

    CClientDC dc(this);
    CFont * cf = GetFont();
    if (cf)
        dc.SelectObject(cf);
    dc.SetBkMode(OPAQUE);
    dc.SetBkColor(GetSysColor(COLOR_MENU));

    if (ks[VK_CAPITAL] & 1)
        dc.SetTextColor(RGB(0, 0, 0));
    else
        dc.SetTextColor(RGB(127,127,127));

    dc.DrawText("CAP", rect, DT_LEFT);
}
```

Notes

6. From **CMainFrame::OnSize**, invoke **CStatusBarCtrl::MoveWindow** to reposition the status bar whenever the frame resizes.

```
if (m_wndStatusBar.m_hWnd)
    m_wndStatusBar.MoveWindow(0, cy, cx, cy, TRUE);
```

7. Override the **OnDraw** member function of the view class to update the status bar each time the user changes the color of the text.

   To see sample code that shows how to override the **OnDraw** function, click this icon.
    (CD-ROM displays the following sample code.)

```
// Updating the view and invalidating the statusbar
// Samples\CH03\CustSBar
void CCustomStatusBarView::OnDraw(CDC* pDC)
{
    CCustomStatusBarDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    CRect r;
    GetClientRect(&r);
    int x = r.right / 2, y = r.bottom / 2;

    pDC->SetTextColor(pDoc->GetColor());
    pDC->SetTextAlign (TA_CENTER | TA_BASELINE);
    pDC->TextOut (x, y, pDoc->GetPhrase());

    //Invalidate status bar to enable repainting
    CMainFrame * cmf = (CMainFrame *)GetParent();
    UINT mID;
    switch(pDoc->GetColor())
    {
        case RED:
            mID = IDM_REDBITMAP;
            break;
        case GREEN:
            mID = IDM_GREENBITMAP;
            break;
        case BLUE:
            mID = IDM_BLUEBITMAP;
            break;
        default:
            mID = IDM_BLACKBITMAP;
    }
    cmf->pt.bitmapID = mID;

    // Force the status bar to be redrawn.
    cmf->m_wndStatusBar.Invalidate();

}
```

Notes

The code samples discussed in this topic appear in the sample application CustSBar, which is located in the folder \Samples\Ch03.

Notes

53 Enhancing Dialog Boxes

Visual Studio provides a dialog box template that serves as the basic interface for a dialog box. By default, the template provides a window with a caption, a **Close** button, and **OK** and **Cancel** buttons. You add controls to the dialog box and code to your application to complete the functionality of the dialog box. In this section, you will learn how to use advanced techniques to extend the functionality of a dialog box.

This section includes the following topics:

- Customizing Common Dialog Boxes
- Extending Dialog Data Validation
- Coordinating Controls
- Creating Property Sheets and Tabbed Dialog Boxes

Notes

54 Customizing Common Dialog Boxes

A common dialog box is a system-defined dialog box that users can use to perform complex operations that are common to most applications. You use common dialog boxes to make the user interface of your applications consistent with other Windows-based applications. Common dialog boxes supported by MFC include **Color**, **Print**, **Open**, and **Save As**.

For information about common dialog boxes, search for "Common Dialog Boxes" in MSDN Library Visual Studio 6.0.

You can customize a common dialog box by altering its layout or behavior to meet the requirements of your application.

Customizing a Common Dialog Box

The following general steps describe how to customize a common dialog box.

1. Create a custom dialog box template.

   Note For information about how to customize a common dialog box template, refer to Knowledge Base article Q132909.

    (CD-ROM displays the following tip.)

   The Microsoft Knowledge Base contains detailed technical articles and samples that are created and maintained by Microsoft Product Support Services (PSS). This database is updated daily and contains technical information about Microsoft products, fix lists, documentation errors, and answers to commonly asked technical support questions.

   To search for Knowledge Base articles on the Microsoft Technical Support Web site, go to the topic Knowledge Base Articles in the Library.

2. Derive a class for the dialog box template from a common dialog box class. For example, the following statement derives a class from **CColorDialog**:

   ```
   class MyColor : public CColorDialog
   ```

   Note Hide or disable controls in the common dialog boxes rather than deleting them.

3. Add a message map to process notification messages:
   - Add handlers for any new added controls.
   - Pass unprocessed messages to the base class.
4. Create a common dialog box object of the new class and implement it using the **DoModal** function.

Notes

Customization Complexity

A common dialog box may require significant changes before you can use it in your application, or it may require only a few minor adjustments. How you customize a common dialog box depends on the complexity of the changes you want to make.

Creating a New Template

If major changes are needed, consider starting with the dialog box template and making whatever changes you need—adding controls, deleting controls, and deriving a unique class from **CDialog**. You can make these changes to the template, but the resulting dialog box will not be a common dialog box.

Making Minor Behavioral Changes to a Common Dialog Box

To change only the behavior of a common dialog box, first consult the Windows Software Development Kit (SDK). You can customize each common dialog box class to perform in different ways, depending on flags or settings in the construction of the object. For example, you can customize the **File Open** dialog box to hide files that are read-only.

Changing Layout and Behavior of a Common Dialog Box

Between the two levels of changes above are less significant changes to the dialog box layout and behavior. For example, if the user of a file-based application is unable to create new folders, you can safely edit the File.dlg resource and hide that particular button.

Notes

55 Extending Dialog Data Validation

ClassWizard implements standard DDV by adding code inside the block marked `//AFX_DATA_MAP`. For example, ClassWizard inserts validation statements to verify the length of an edit control string.

If you want your application to include validation capabilities that are not part of standard DDV, such as checking whether an edit control contains data, you will need to write the appropriate code.

▶ To extend DDV

1. Check the value of the **pDX->m_bSaveAndValidate** member variable. If it is **TRUE**, DDV is occurring.
2. If DDV is occurring, check the data variables in the dialog object.
3. If the value of the data variable is not within the proper range:
   a. Use **::AfxMessageBox** to display an error message.
   b. Call **pDX->Fail** to exit **DoDataExchange**.

To see sample code that shows how to extend DDV, click this icon.
 (CD-ROM displays the following sample code.)

```
// Enhancing DDV
// Samples\Ch03\Modeless
if(pDX->m_bSaveAndValidate)
{
    if (0 == m_strDefault.GetLength())
    {
        AfxMessageBox("Default phrase cannot be empty");
        DDX_Text(pDX, IDC_EDIT1, m_strDefault);
        pDX->Fail();
    }
    // Check other dialog class variables in a similar
    // manner.
}
```

For more information about extending DDV, search for "CDataExchange" in MSDN Library Visual Studio 6.0.

Notes

56 Coordinating Controls

In response to user interaction with a control, you can change the text in a static control or on a button. This requires that you create a command UI handler. For example, you can create a command UI handler to enable or disable controls, or to change the value displayed in a control.

Enabling and Disabling Controls

The **Apply** button is an example of a control that you can enable or disable in a dialog box. You disable the **Apply** button initially in the **OnInitDialog** function and enable the **Apply** button when the user changes the text in a text box.

▶ **To enable or disable a control**

1. Use the **CWnd::GetDlgItem** function to get a pointer to the control.
2. Use the **CWnd::EnableWindow** function to enable or disable the control.

The following example code enables the **Apply** button:

```
void CModelessDlg::OnChangeMlPhrase()
{
    GetDlgItem(IDC_APPLY)->EnableWindow(TRUE);
}
```

Changing the Value Displayed in a Control

The **Find** dialog box is an example of a situation where you may want to change the text displayed in a control. After the first instance of a string has been found, you can adjust the text on the search button to read **Find Again**, rather than **Find**.

▶ **To change the value displayed in a control**

- Use the **CWnd::SetDlgItemText** function to change the text that is displayed in a control.

  –or–

- Use the **CWnd::SetDlgItemInt** function to change the numeric value that is displayed in a control.

The following example code shows how you can use the **SetDlgItemInt** function to modify the numeric value of a field:

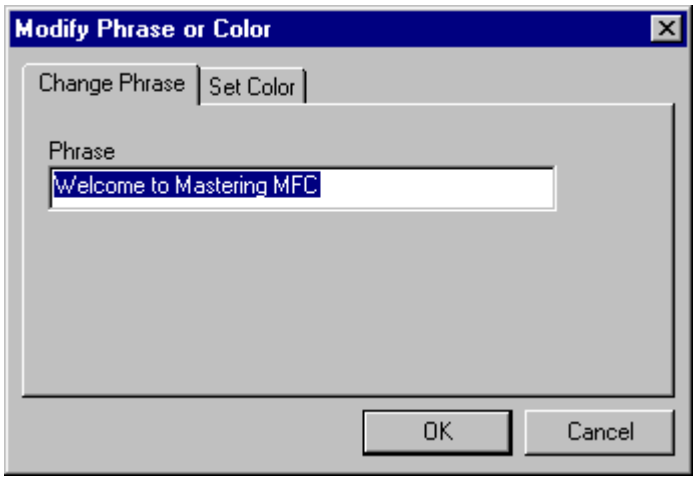
```
void CListWithItemDataDlg::OnIncrement()
{
    int m_nIncreasedValue;
     ...
     SetDlgItemInt(IDC_EDIT1, m_nIncreasedValue);
     ...
}
```

Notes

58 Creating Property Sheets and Tabbed Dialog Boxes

A property sheet is generally used to modify the attributes of an external object, such as the current selection in a view. Often, a property sheet is a dialog box that is made up of tabbed pages, called property pages. These pages are displayed one at a time as the user clicks the tabs at the top of the dialog box. You can use a tabbed property sheet to display a large amount of information in an organized manner.

To see an illustration of a tabbed property sheet, click this icon.
 (CD-ROM displays the following illustration.)



A property sheet has three main parts:

- The dialog box
- One or more property pages
- The tabs at the top of each property page that the user clicks to select the page.

For style guidelines about using property sheets in your application, search for "Guidelines, User-Interface" in MSDN Library Visual Studio 6.0.

If you do not want to use the default **CPropertyPage** object, create a dialog box by using the dialog editor. Then create a class for it, and derive the class from **CPropertyPage**. Use the derived class instead of **CPropertyPage** in the procedures that follow.

▶ **To create property pages**

1. In ResourceView, right-click the Dialog folder and click **Insert**.
2. Expand the Dialog folder and double-click **IDD_PROPPAGE_LARGE** (or **IDD_PROPPAGE_MEDIUM** or **IDD_PROPPAGE_SMALL**) to invoke the dialog bar editor.
3. Add controls to the property page.
4. Create a class for each dialog box that is derived from **CPropertyPage**. ClassWizard automates this process, but be sure to set the base class to **CPropertyPage**.

Notes

▶ **To create a property sheet**

1. Create the property page objects.
2. Create a **CPropertySheet** object that will contain the property page objects.
3. Add the property page objects. Add the largest page first because this will set the overall size of the dialog box. The tabs that appear at the top of each property sheet are inserted by the **CPropertySheet::AddPage** method. For details, see the sample code at the end of this topic.
4. If you want the property sheet to be deployed as a modal dialog box, use the **DoModal** function.

Note If the user clicks **OK**, the property sheet will extract the values from the individual property page objects.

Notes

To see sample code that creates a property sheet, click this icon.
 (CD-ROM displays the following sample code.)

```
//Creating and displaying a tabbed dialog box
//Samples\CH03\Tabbed
void CTabbedColorPhraseView::OnModifyShowtabbeddialogbox()
{
    CTabbedColorPhraseDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    // Create each property page object and initialize
    // as appropriate.
    CPhraseTab phraseTab;
    phraseTab.m_phrase = pDoc->GetPhrase();

    CColorTab colorTab;
    colorTab.m_color = pDoc->GetColor();

    // Create the property sheet object and give it a
    // title.
    CPropertySheet cps("Modify Phrase or Color");

    // Add the largest property page first.
    cps.AddPage(&phraseTab);
    cps.AddPage(&colorTab);

    // Since this is being displayed modally instead of
    // modelessly, remove the Apply button, which
    // appears by default on Property Sheets.
    cps.m_psh.dwFlags |= PSH_NOAPPLYNOW;

    // If the user clicks on OK and either color or
    // phrase has changed, update the document and
    // invalidate the view.
    if (IDOK == cps.DoModal())
        if (pDoc->GetPhrase() != phraseTab.m_phrase ||
             pDoc->GetColor() != colorTab.m_color)
        {
            pDoc->SetColor(colorTab.m_color);
            pDoc->SetPhrase(phraseTab.m_phrase);
            Invalidate();
        }
}
```

The code samples discussed in this topic appear in the sample application Tabbed, which is located in the folder \Samples\Ch03.

Notes

61 Using Modeless Dialog Boxes

You create and test modeless dialog boxes in basically the same way that you do modal dialog boxes. However, modeless dialog boxes require some additional work.

This section shows you how to create a persistent object, create and display the modeless dialog box, implement the **OK** and **Cancel** buttons, and destroy the dialog box when the user finishes with it. You will also learn how to establish communication between the dialog box and the parent application by using the **RegisterWindowMessage** function.

This section includes the following topics:

- Initializing and Displaying a Modeless Dialog Box
- Implementing Command Buttons
- Deleting the Dialog Box Object
- Communicating with the Application

Notes

62 Initializing and Displaying a Modeless Dialog Box

This topic describes how to create and initialize a modeless dialog box. Notice that the first two steps in the process—creating and testing the dialog box template and creating the class—are the same as with modal dialog boxes.

▶ **To initialize and display a modeless dialog box**

1. Use the dialog editor to create and test the dialog box template.
2. Use ClassWizard to create the dialog box class and add DDX and DDV support as appropriate.
3. Declare a persistent dialog box object.

   To ensure that it will be available when needed, make the dialog box object (or pointer) an embedded member of the application class or the main window. If you use a pointer, you invoke the **New** operator and the resulting constructor to allocate and initialize storage for the object.
4. Create the dialog box object by using the **CDialog::Create** message handler.

   The first argument to **Create** is the name or the ID of the dialog box template. The second argument is a pointer to the dialog object's parent window. If the pointer is NULL, the main application window becomes the parent.

   The call to **Create** can either be in the constructor, or immediately after the invocation of the constructor.

Notes

To see sample code that shows how to create a modeless dialog box, click this icon.
 (CD-ROM displays the following sample code.)

```
//Creating and displaying a modeless dialog box
//Samples\CH03\Modeless
class CModelessView : public CView
{
...
protected:
    // CListWithItemDataDlg is the class derived from CDialog
    // for the modeless dialog box
    CListWithItemDataDlg * m_pDlg;
...
};

void CModelessView::OnModifyDisplaymodelessdialog()
{
    CModelessDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    // Create and initialize the dialog box object.
    // It must be persistent (not stack-based).
    m_pDlg = new CListWithItemDataDlg(this);
    m_pDlg->m_color = pDoc->GetColor();
    m_pDlg->m_phrase = pDoc->GetPhrase();
    m_pDlg->Create(IDD_PHRASE_MODELESS);

    // The call to Create has caused DDX to occur.
    // Everything's done, show the window. Note that the
    // DB itself has its visible property set to false
    // in the resource editor.
    m_pDlg->ShowWindow(SW_RESTORE);
}
```

In the sample code, the dialog box is persistent rather than stack-based.

5. Display the dialog box.

   If the dialog box has the style property WS_VISIBLE, it will appear immediately. You can control the dialog box's visibility by calling the **CWnd::ShowWindow** function with the parameters SW_SHOW, SW_HIDE, and SW_RESTORE, respectively.

To see a demonstration of how to create a modeless dialog box and insert data, click this icon.
 (CD-ROM plays the demonstration, "Creating a Modeless Dialog Box.")

Notes

63 Implementing Command Buttons

In a modeless dialog box, you can replace the **OK** button and override the **Cancel** button. Unlike the default version of the **OnCancel** message handler, your override of **OnCancel** should not call the **EndDialog** function to terminate the dialog box. Instead, call the **CWnd::DestroyWindow** function to terminate the modeless dialog box.

The following example code terminates a modeless dialog box:

```
void CListWithItemDataDlg::OnCancel()
{
    m_pWnd->SendMessage(m_UserMsg, ML_CANCEL);
    DestroyWindow();
}
```

Notes

64 Deleting the Dialog Box Object

To delete a dialog box object, override the **PostNcDestroy** (where **Nc** stands for nonclient) function. Note that the owner of the dialog box (the application that created it) probably has a pointer that will not be valid after the modeless dialog box is destroyed.

The following example code deletes a persistent dialog box object:

```
void CListWithItemDataDlg::PostNcDestroy()
{
    delete this;
    CDialog::PostNcDestroy();
}
```

Notes

65 Communicating with the Application

You must create a communication link between the dialog box and its parent application for modeless dialog boxes. You can use the **RegisterWindowMessage** and **SendMessage** functions to create this communication link.

To see a demonstration of how to use the **RegisterWindowMessage** function to implement communication between a modeless dialog box and its parent application, click this icon.

 (CD-ROM plays the demonstration, "Implementing Dialog Box Communication.")

To see sample code that shows how to enable communication by using **RegisterWindowMessage**, click this icon.

 (CD-ROM displays the following sample code.)

```
// Handlers to implement communciation between a modeless dialog box
// and its parent window
// Samples\CH03\Modeless
// Place manifest constants in the modeless
// dialog class's header file for the registered
// message and for each message that the dialog
// box will communicate back to the CWnd-derived
// object that created it.
#define MY_DIALOGBOX_MSG "LWIDMsg"
#define ML_APPLY 1
#define ML_CANCEL 2

// Overload the dialog box's constructor to
// take a pointer to a CWnd object, and register
// the message in the constructor.
class CListWithItemDataDlg : public CDialog
{
    ...
public:
    CListWithItemDataDlg(CWnd *pWnd,
        CWnd* pParent = NULL);
    ...

protected:
    CWnd * m_pWnd;
    UINT m_UserMsg;
    ...
};
```

(Code continues on following page.)

Notes

```
CListWithItemDataDlg::CListWithItemDataDlg(CWnd * pWnd,
        CWnd* pParent)
    : CDialog(CListWithItemDataDlg::IDD, pParent),
    m_pWnd (pWnd)
{
    //{{AFX_DATA_INIT(CListWithItemDataDlg)
    m_phrase = _T("");
    //}}AFX_DATA_INIT
    m_UserMsg = RegisterWindowMessage(MY_DIALOGBOX_MSG);
}
// From the dialog box's buttons, send a
// message to the object that launched the dialog
// box. Extra information may be passed in the
// third argument of the call to SendMessage.
void CListWithItemDataDlg::OnApply()
{
    UpdateData(TRUE);
    m_pWnd->SendMessage(m_UserMsg, ML_APPLY);

    // Now that the Apply button's work is done,
    // disable it.
    CButton * cb = (CButton *)GetDlgItem(IDC_APPLY);
    cb->EnableWindow(FALSE);
}

void CListWithItemDataDlg::OnCancel()
{
    m_pWnd->SendMessage(m_UserMsg, ML_CANCEL);
    DestroyWindow();
}
// Include the dialog class's header file
// in the class that will receive messages from
// the modeless dialog box, and then register a
// message. In this example, the application's
// view class receives the messages.
// The class of this object must be derived from
// CWnd, since it will be receiving messages. This
// rules out anything derived from CDocument.
// Static member variables must be initialized.
// Register the message the dialog box will send to
// this object.
UINT CModelessView::m_UserMsg =
    RegisterWindowMessage(MY_DIALOGBOX_MSG);
```

(Code continues on following page.)

Notes

```
// Add  an entry to the class's message map.
// (You must do this manually; ClassWizard can't do
// this for you.)
BEGIN_MESSAGE_MAP(CModelessView, CView)
    //{{AFX_MSG_MAP(CModelessView)
    ON_COMMAND(...
    ON_UPDATE_COMMAND_UI(...
    //}}AFX_MSG_MAP
    ON_REGISTERED_MESSAGE(CModelessView::m_UserMsg, MyMessageHandler)
END_MESSAGE_MAP()

// Create the dialog box and pass in a
// pointer to the object.
class CModelessView : public CView
{
    ...
protected:
    CListWithItemDataDlg * m_pDlg;
    ...
};

void CModelessView::OnModifyDisplaymodelessdialog()
{
    ...
    // Create and initialize the dialog box object.
    // It must be persistent (not stack-based).
    m_pDlg = new CListWithItemDataDlg(this);
    ...
}
// In the view's message handler, respond
// to the messages.
```

(Code continues on following page.)

Notes

```
LRESULT CModelessView::MyMessageHandler(WPARAM wParam, LPARAM lParam)
{
    CModelessDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    switch (wParam)
    {
        case ML_APPLY:
            pDoc->SetPhrase(m_pDlg->m_phrase);
            pDoc->SetColor(m_pDlg->m_color);
            Invalidate();
            break;

        case ML_CANCEL:
            // The dialog box's own code will properly
            // destroy the object, but it's important
            // that this class set its pointer to the
            // dialog box to 0.
            m_pDlg = 0;
    }
    return TRUE;
}
```

Notes

66 Using Dialog Bars

In this section you will learn about dialog bars and look at the steps in adding a dialog bar to your application.

This section includes the following topics:

- Introduction to Dialog Bars
- Adding a Dialog Bar
- Implementing Command Handlers

Notes

66 Introduction to Dialog Bars

A dialog bar is a control bar with toolbar and dialog box characteristics. Because it is based on a dialog template and has the attributes of a modeless dialog box, a dialog bar can contain any kind of control that a dialog box can contain, but it behaves like a powerful toolbar.

The following illustration shows a dialog bar with a variety of controls.



MFC supports dialog bars with the class **CDialogBar**. For more information about **CDialogBar**, search for "CDialogBar" in MSDN Library Visual Studio 6.0.

Notes

68 Adding a Dialog Bar

Creating and implementing a dialog bar involves several steps:

1. Add a menu item to control the display of the dialog bar.
2. Create dialog bar resources.
3. Attach the dialog bar to the **CMainFrame** class.
4. Create a **CDialogBar** class.

These steps are described in more detail below.

To see a demonstration of how to create and implement a dialog bar, click this icon.
 (CD-ROM plays the demonstration, "Creating and Implementing a Dialog Bar.")

▶ **To add a menu item to control the dialog bar**

1. Open the application in which you want to insert the dialog bar.
2. In ResourceView, expand the Menu folder.
3. Double-click **IDR_MAINFRAME** to invoke the menu editor.
4. In the menu editor, click **View**, and then double-click the empty box at the bottom of the drop-down menu (beneath the **Status Bar** option) to display the **Menu Item Properties** dialog box.
5. In the **Menu Item Properties** dialog box, set the following properties as shown.

Property	Value
ID	ID_VIEW_DIALOGBAR=0xE821
Caption	&Dialog Bar
Prompt	Show or hide the dialog bar\nToggle Dialog Bar

Note Setting ID to ID_VIEW_DIALOGBAR=0xE821 causes the associated resources to be used in calculating the layout of control bars. For a more detailed explanation of toolbar layout, search for "Control Bars" in MSDN Library Visual Studio 6.0.

▶ **To create the dialog box resources**

1. In ResourceView, right-click the Dialog folder and click **Insert**.
2. Expand the Dialog icon and double-click **IDD_DIALOGBAR** to invoke the dialog bar editor.
3. Delete the static **TODO** control from the dialog bar.
4. Drag the controls you want to add from the control palette to the dialog bar.
5. Optionally add a **Picture** control, setting properties as follows:
   - Set **Type** to **Icon**.
   - Set **Image** to the ID of an icon.
   - Set **Notify** to **True**.
   - Set **ID** to the ID of its corresponding menu item.

Notes

▶ **To attach the dialog bar to the CMainFrame class**

1. Start ClassWizard when you have finished designing the dialog bar, but before you save it.

   You will not build a new class for the control bar. Instead, you will associate the control bar with the class **CMainFrame**. This way, ClassWizard has access to the identifiers in the dialog bar so it can generate message map entries in an existing class. Without this step, you would have to edit the message map yourself.
2. In **ClassWizard**, in the **Adding a Class** dialog box, click **Select an existing class**, and then click **OK**.
3. In the **Select Class** dialog box, select the **CMainFrame** class.

   If the dialog bar's identifiers are available to the **CMainFrame** class, ClassWizard will also make them available to the view class.
4. Click **Select**, click **Yes** in the displayed message box, and then click **OK**.

▶ **To create a CDialogBar object in the CMainFrame class**

1. Right-click **CMainFrame** in ClassView and add a protected member variable as follows:

   ```
   CDialogBar m_wndDialogBar
   ```
2. In **CMainFrame::OnCreate**, attach the dialog bar to the **CDialogBar** class. Specify the control ID of the dialog bar as the last argument. For example:

   ```
   m_wndDialogBar.Create(this, IDD_DIALOG_BAR, CBRS_TOP, ID_VIEW_MYDIALOGBAR);
   ```

   The first argument, `this`, is a handle to the window that owns the dialog bar. In this case, it is the **CMainFrame** object. The second argument, `IDD_DIALOG_BAR`, is the ID of the dialog box. The third argument, `CBRS_TOP`, defines how the dialog bar will be aligned in the associated window. The fourth argument, `ID_VIEW_MYDIALOGBAR`, is the control ID for the dialog bar.
3. Initialize the dialog bar and its controls as follows:

   ```
   CComboBox * p_lb =
       (CComboBox *)m_wndDialogBar.GetDlgItem(IDC_LIST_COLORS);
   p_lb->SetCurSel(0);
   ```

Notes

69 Implementing Command Handlers

A command handler processes an event from a menu item and calls the appropriate function based on the event.

▶ **To add a command handler for any list box or combo box in the dialog bar**

1. Open the dialog bar resource.
2. Invoke ClassWizard and add a command handler for any list boxes or combo boxes. Typically you need to implement the event LBN_SEL_CHANGE or CBN_SEL_CHANGE. You can choose to place the command handler in the most convenient place. The view class is a good choice, because the list box or combo box probably affects the document, and most command handlers that affect the document are placed in this class.
3. Implement the command handler. The following example code shows a sample implementation:

```
void CDlgBarsView::OnSelchangeListColors()
{
    CMainFrame * cmf = (CMainFrame *)AfxGetMainWnd();
    CComboBox * p_cb =(CComboBox *)
        cmf->m_wndDialogBar.GetDlgItem(IDC_LIST_COLORS);

    // respond to user changing selection in combo box
}
```

▶ **To add command and command UI handlers to the** CMainFrame **class**

1. Use ClassWizard to add command and command UI handlers to the **CMainFrame** class for the **View Dialog** menu item.
2. Add code for the **OnViewDialogbar** and **OnUpdateViewDialogbar** functions that were created when you added command handlers. For example:

```
void CMainFrame::OnViewDialogbar()
{
    OnBarCheck(ID_VIEW_MYDIALOGBAR);
}

void CMainFrame::OnUpdateViewDialogbar(CCmdUI* pCmdUI)
{
    OnUpdateControlBarMenu(pCmdUI);
}
```

Note The functions **OnBarCheck** and **OnUpdateControlBarMenu** are members of the **CFrameWnd** class, even though they are not documented in MSDN Library Visual Studio 6.0.

In general, implementing command buttons or icons on a dialog bar is easy. You create a menu item that performs the same operation, and then assign the button or icon on the dialog bar the same identifier as the menu. The menu item can be a dummy menu item, but it must have a handler.

You can also add a single dialog bar from Component Gallery. When you use this method, the Component and Controls Gallery provides most of the code, but you still must do some work.

Notes

You can implement multiple dialog bars in an application. You also can create dialog bars dynamically. When doing so, the parent of a dialog bar, which is passed in as the first argument, must be **CMainFrame**. In most cases, it is sufficient to create dialog bars as described in this section.

Notes

70 Using Rebars

A rebar object is very similar in behavior to a toolbar object. It acts as a container for child windows such as edit boxes, toolbars, dialog bars, and list boxes.

In this section, you will learn to create and customize rebars.

This section includes the following topics:

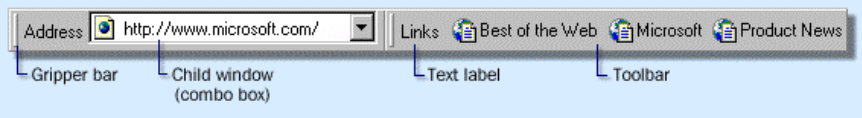
- Introduction to Rebars
- Creating a Rebar
- Adding Child Windows to a Rebar
- Customizing Rebars

Notes

71 Introduction to Rebars

A rebar control can contain one or more bands, with each band having any combination of a gripper bar, a bitmap, a text label, and a child window. However, bands cannot contain more than one child window.

The following illustration shows a rebar control that has two bands. One band contains a gripper bar, a text label ("Address"), and a combo box child window. The other band contains a gripper bar, a text label, and a flat toolbar (implemented with a child window).



Your application can automatically resize the rebar, or the user can manually resize the rebar by clicking or dragging its gripper bar.

Rebars do not support MFC control-bar docking. If the **CRebar::EnableDocking** function is called, your application will display an error message.

Rebar Classes

MFC provides two classes for creating rebars: **CReBar** and **CReBarCtrl**. The **CReBar** class provides the functionality of the rebar common control, and it handles many of the required common control settings and structures for you. The **CReBar** class implements its functionality by using the **CReBarCtrl** class.

Notes

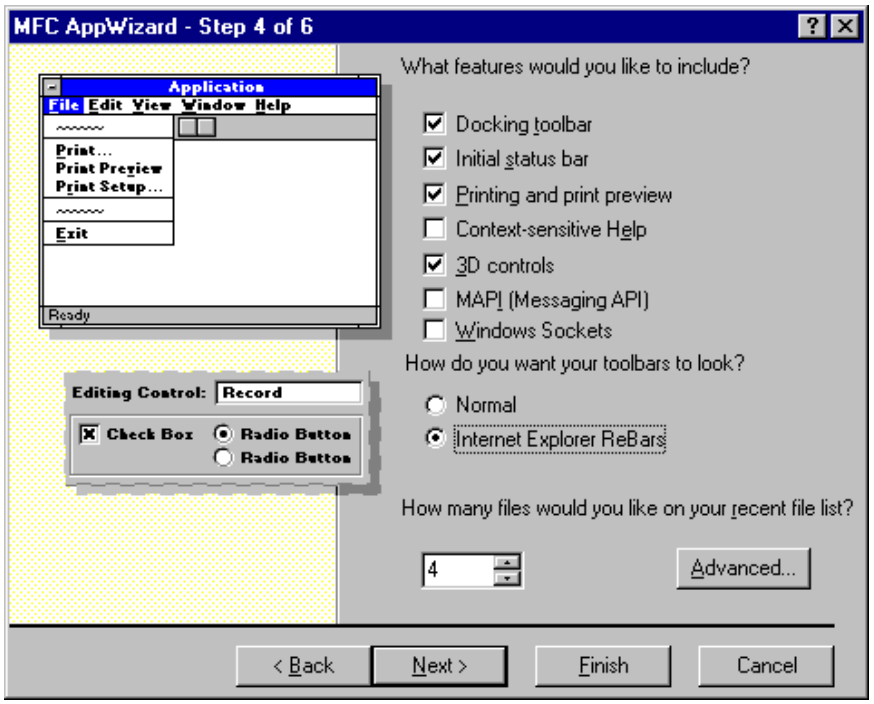
72 Creating a Rebar

You can create a rebar control in two ways. You can either use MFC AppWizard to add the code that will create the rebar at run time or manually add the code to your application to create it.

Creating a Rebar Using MFC AppWizard

You can create a rebar by choosing the **Internet Explorer ReBars** option in Step 4 of **MFC AppWizard**.

To see an illustration of the **MFC AppWizard - Step 4 of 6** dialog box, click this icon.
(CD-ROM displays the following illustration.)



Notes

MFC AppWizard performs the following steps to create a rebar control:

- Defines a protected **CReBar** object as follows:

```
CReBar    m_wndReBar;
```

- Includes the following lines of code in the **OnCreate** function of the **MainFrame** class:

```
if (m_wndReBar.Create(this))
    {
        if(m_wndReBar.AddBar(&m_wndToolBar) &&
             m_wndReBar.AddBar(&m_wndDlgBar))
            {...} //Success
        else
            {...} // Failed to add controls
    }
else
    {...} //Failed to create the rebar
```

  The **Create** function creates a rebar control and attaches it to the **CReBar** object. Apart from a pointer to the parent window—normally your frame window—the **Create** function takes parameters such as the rebar control style, the rebar window styles, and the rebar's child-window ID.

  For more information about **CReBar::Create**, search for "CReBar class members" in MSDN Library Visual Studio 6.0.

Creating a Rebar Without Using MFC AppWizard

You can add rebar functionality to an application by defining a protected **CReBar** object and invoking the **CReBar::Create** function from within the **OnCreate** function of the **MainFrame** class. You can add existing controls, such as toolbars and dialog bars, by using the **CReBar::AddBar** function. The next topic discusses this function in more detail.

Notes

73 Adding Child Windows to a Rebar

After creating a rebar control, you call the **CReBar::AddBar** function to add controls such as toolbars and dialog bars to a rebar. The function first creates a rebar band and then adds the control to the band.

The following example code shows how to use the **AddBar** member function:

```
if (!m_wndReBar.AddBar(&m_wndToolBar) ||     !m_wndReBar.AddBar(&m_wndDlgBar))
    {
        TRACE0("Failed to add controls to the rebar\n");
        return -1;      // failed to create
    }
```

For more information about **CReBar::AddBar**, search for "AddBar" in MSDN Library Visual Studio 6.0.

Notes

74 Customizing Rebars

You can modify the default values of a rebar by using the **CReBar::GetReBarCtrl** member function, which provides direct access to the underlying rebar control. The function returns a reference to a **CReBarCtrl** object. This enables you to use the member functions of the **CReBarCtrl** class.

For example, call the **SetBkColor** member function of **CReBarCtrl** to change the background color of the rebar, as shown in the following example code:

```
m_wndReBar.GetReBarCtrl().SetBkColor(RGB(0,0,255));
```

Notes

76 Sample Applications

The following table describes the sample applications related to this chapter. These sample applications are located in the folder \Samples\Ch03.

Sample application	This sample application shows you how to:
Dynamenu	Manage menus dynamically.
Owndraw	Implement ownerdraw menus by drawing menus that include bitmaps.
Custsbar	Replace the default status bar with a custom status bar. This custom status bar duplicates the functionality of the default status bar (CAPS state, status line) and has graphics drawn in one of its panes.
Toolbar	Add a floating, dockable toolbar.
Tabbed	Implement a modal tabbed dialog box with three property sheets.
Modeless	Implement a modeless dialog box.
Dlgbars	Add a dialog bar featuring buttons, icons, and combo boxes.

Notes

80 Self-Check Questions

1. Which function adds menu items to a cascading menu?

- A. **AppendMenuItems**
- B. **AddPopupItems**
- C. **AppendMenu**
- D. **AddMenu**

2. To create a dockable toolbar, which steps must you complete?

- A. You only need to enable docking for the frame window.
- B. You only need to enable docking for each toolbar you want to be dockable.
- C. You only need to add code to dock the toolbar to a frame window.
- D. You must complete all three steps: enable docking for the frame, enable docking for each toolbar you want to be dockable, and dock the toolbar to the frame.

3. Which drawing type must you specify to include graphics in a status bar?

- A. SBT_IMAGES
- B. SBT_OWNERDRAW
- C. SBT_NOBORDERS
- D. SBT_POPOUT

4. How do you invoke modal and modeless dialog boxes?

- A. Invoke modal dialog boxes with **CDialog::DoModal**. Invoke modeless dialog boxes with **CDialog::Create**.
- B. Invoke modal dialog boxes with **CDialog::DoModal**. Invoke modeless dialog boxes with **CDialog::DoModeless**.
- C. Invoke modal dialog boxes with **CModal::Create**. Invoke modeless dialog boxes with **CModeless::Create**.
- D. Invoke modal dialog boxes with **CDialog::Create**. Invoke modeless dialog boxes with **CDialog::DoModal**.

Notes

5. What is the advantage of using common dialog boxes?

- A. Makes your code portable across operating systems.
- B. Improves run-time performance of your applications.
- C. Reduces time needed to build property sheets and tabbed dialog boxes.
- D. Makes the user interface of your applications consistent with other Windows-based applications.

6. Which is the base class of a dialog bar control?

- A. **CDlgCtrl**
- B. **CReBar**
- C. **CDialog**
- D. **CDialogBar**

7. Which function adds controls to a rebar?

- A. **SetReBarCtrl**()
- B. **AddReBar**()
- C. **AddBar**()
- D. **GetReBarCtrl**()

Notes